

Entregable Laboratorio 5 - Seguridad

Proyecto: Fitbeat
Fecha: 11 de mayo de 2026

3.1. Deliverable

Nombres Completos:

- Nicolas Felipe Arciniegas Lizarazo
- Karen Lorena Guzman Del Rio
- Juan David Chacon Muñoz
- Adrian Yebid Rincon
- Pablo Felipe Sandoval Menjura
- Julio Cesar Albadan Sarmiento

1. Architectural View (Vista de Componentes y Conectores)

Ubicación del Patrón: El patrón **Secure Channel** se aplica en el conector externo que vincula a los clientes (**Web Frontend / CLI**) con el **API Gateway (Traefik)**.

En esta vista, el conector ha pasado de ser un canal simple de tipo *Request/Reply (HTTP)* a un **Secure Request/Reply (HTTPS/TLS 1.2+)**. Traefik actúa como el "Punto de Terminación TLS", lo que significa que el canal seguro se cierra en la frontera del sistema, permitiendo que el tráfico interno hacia los microservicios fluya en una red privada protegida.

2. Technical Guide (Guía Técnica)

2.1. Description of the pattern and its purpose

El **Secure Channel Pattern** tiene como propósito establecer un túnel de comunicación cifrado entre dos entidades para garantizar que la información sensible no pueda ser interceptada ni modificada por terceros. Su implementación asegura que los extremos de la comunicación estén plenamente identificados mediante certificados digitales, mitigando ataques de *Man-in-the-Middle (MitM)*.

2.2. Quality scenario addressed

Escenario de Autenticación:

"Cuando un usuario envía sus credenciales (correo electrónico y contraseña) desde la CLI o la Web hacia el sistema Fitbeat para iniciar sesión, los datos deben viajar cifrados en un canal seguro. El sistema debe garantizar la **confidencialidad** para que ningún atacante con acceso a la red pueda leer las credenciales en texto plano, y la **integridad** para asegurar que la solicitud de inicio de sesión no sea alterada en tránsito."

2.3. Steps followed to implement the pattern

Paso 0: Preparación de Herramientas

- OpenSSL:** Instalado mediante Git Bash (Windows) para actuar como nuestra fábrica de certificados.
- Librerías:** No se requirieron librerías externas adicionales, aprovechando la capacidad nativa de **Traefik** como Reverse Proxy para la gestión de TLS.

Paso 1: Definición del Escenario de Calidad

Se identificó el proceso de login como el punto más crítico. El canal seguro debe proteger el envío de credenciales hacia el `auth-service` a través del Gateway.

Paso 2: Creación de la Entidad Certificadora (CA) de Fitbeat

- Se creó el directorio de trabajo:

```
mkdir laboratory-5 && cd laboratory-5
```

- Generación de la Llave Privada de la CA:**

```
openssl genrsa -out fitbeat-ca.key 2048
```

- Generación del Certificado Raíz (Root Certificate):**

```
MSYS_NO_PATHCONV=1 openssl req -x509 -new -nodes -key fitbeat-ca.key -sha256 -days 365 \
-out fitbeat-ca.crt -subj "/CN=Fitbeat-CA/O=Universidad Nacional/C=CO"
```

Configuración: En "Common Name", se asignó `Fitbeat Authority`.

Paso 3: Generación del Certificado para el "Secure Channel"

1. Generación de la llave privada del servidor:

```
openssl genrsa -out fitbeat-server.key 2048
```

2. Creación del CSR (Certificate Signing Request):

```
MSYS_NO_PATHCONV=1 openssl req -new -key fitbeat-server.key -out fitbeat-server.csr \
-subj "/CN=localhost/O=Fitbeat-Project/C=CO"
```

IMPORTANTE: En "Common Name", se definió `localhost`.

3. Firma del certificado con la CA:

```
openssl x509 -req -in fitbeat-server.csr \
-CA fitbeat-ca.crt -CAkey fitbeat-ca.key \
-CAcreateserial -out fitbeat-server.crt -days 365 -sha256
```

2.4. Configuration and code snippets used

A. Configuración de Infraestructura (`docker-compose.yml`)

Se inyectaron los certificados y se habilitó el puerto 443 para el tráfico HTTPS.

```
ports:
  - "8090:80"    # HTTP Inseguro (Para pruebas de pentest)
  - "443:443"    # HTTPS Seguro (Canal establecido)
volumes:
  - ./laboratory-5/certs:/etc/traefik/certs:ro # Certificados en modo solo lectura
```

B. Configuración del Canal Seguro (`traefik/dynamic.yml`)

```
tls:
  certificates:
    - certFile: "/etc/traefik/certs/fitbeat-server.crt"
      keyFile: "/etc/traefik/certs/fitbeat-server.key"
  options:
    default:
      minVersion: VersionTLS12
      cipherSuites:
        - TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
        - TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA256
```

2.5. Results and improvements observed (Pentest con Wireshark)

Se realizó una comparación técnica interceptando el tráfico durante el proceso de autenticación:

1. **Sin Canal Seguro (Puerto 8090):** Al capturar el tráfico HTTP, se observó el paquete POST de login con el JSON de credenciales (`email` y `password`) en texto plano, totalmente legible para cualquier interceptor.

(Aquí se adjunta la captura de pantalla de Wireshark mostrando el texto plano.)

2. **Con Canal Seguro (Puerto 443):** Al repetir el proceso vía HTTPS, Wireshark solo capturó el *TLS Handshake*. Toda la información de autenticación viajó cifrada bajo el protocolo TLS 1.2, resultando en datos ininteligibles (gibberish) para el atacante.

(Aquí se adjunta la captura de pantalla de Wireshark mostrando el tráfico Application Data cifrado.)

2.6. Recommendations for other teams

- **Uso de `MSYS_NO_PATHCONV`:** Al trabajar en Windows con Git Bash, es vital usar esta variable de entorno al generar el CSR para evitar que las barras diagonales del parámetro `-subj` sean convertidas en rutas de archivos:

```
MSYS_NO_PATHCONV=1 openssl req ...
```

- **Terminación TLS vs End-to-End:** En entornos internos altamente sensibles, se recomienda llevar el canal seguro hasta el microservicio, aunque para la mayoría de arquitecturas de red privada, la terminación en el Gateway es el equilibrio óptimo entre seguridad y rendimiento.
- **Volúmenes Solo Lectura:** Siempre montar los certificados con el sufijo `:ro` para evitar que procesos dentro del contenedor puedan manipular la identidad del servidor.